From Software Models to Optical Circuits: Operator-Level Translation of Convolutional Neural Networks for Photonic Neural Processors

Dalton Omondi
Electrical and Electronics Engineering, Kenyatta University, Nairobi, Kenya
Email: daltonomondi588@gmail.com

Abstract

Photonic neural networks (PNNs) provide a promising approach to alleviate the energy and latency limitations of electronic deep learning accelerators. A key challenge, however, lies in systematically embedding trained models into silicon photonic (SiP) hardware. This work introduces an operator-based framework that maps neural network primitives—including convolutions, activations, pooling, and residual connections—onto SiP building blocks such as Mach–Zehnder interferometer meshes, microring resonators, and multimode interference couplers. The framework is expressed through mathematical operators and C/C++ array structures, enabling direct translation from software models to optical circuit layouts. As a case study, MNIST inference is demonstrated using spectrum-slicing convolutions with photodetection pooling, achieving approximately 98% accuracy at 0.1 fJ/MAC and 7.6×10^6 inferences/s. To examine scalability, the methodology is extended to EfficientNet's MBConv block, where depthwise and pointwise layers are implemented using microring filters and reconfigurable MZI cores, and squeeze-and-excitation is realized through hybrid optical–electronic pathways. The results establish a workflow for photonic accelerator design, demonstrating both energy-efficiency potential and involved practical constraints, including accumulated optical losses.

Keywords: Silicon Photonics SiP, Photonic Neural Networks PNN, Convolutional Neural Networks CNN, Optical Accelerators, EfficientNet, Mach–Zehnder Interferometer MZI, Microring Resonator MRR, Photonic Integrated Circuits PIC, Optical Spectrum Slicing OPS, Phase-Change Materials PCM, Depthwise Separable Convolution, Neuromorphic Photonics

1 Introduction

The rapid advancement of deep learning has transformed domains such as computer vision and natural language processing. However, the associated electronic hardware faces increasing challenges in power consumption and computational throughput. As models grow in scale and complexity, conventional von Neumann architectures—even with specialized accelerators such as GPUs and TPUs—remain limited by memory bottlenecks and thermal constraints. Photonic neural networks (PNNs), which exploit the parallelism and speed of optical signal propagation, offer an alternative technology for accelerating inference. Silicon photonics (SiP) enables the integration of photonic components on a single chip. SiP can be fabricated in the existing CMOS foundry, proving a cost-effective pathway toward compact and energy-efficient optical computing platforms.

Despite these advantages, translating trained machine learning (ML) models, such as convolutional neural networks (CNNs), into photonic hardware remains a fragmented process. Current implementations emphasize custom architectures tailored to specific tasks, lacking a generalizable methodology that connects software-level models to hardware-level photonic circuits. This work addresses this limitation by introducing an operator-level translation framework that systematically maps CNN components to SiP primitives. In this approach, CNN operations are formalized as mathematical operators with direct correspondences to photonic building blocks, such as Mach–Zehnder interferometers (MZIs) for linear transformations and microring resonators (MRRs) for spectral filtering. The resulting workflow enables the embedding of arbitrary trained models into photonic processors in a structured and scalable manner.

The remainder of this paper is organized as follows. Section 2 reviews background and motivation, emphasizing the limitations of electronic accelerators and the advantages of photonic approaches. Section 3 formalizes CNN components as mathematical operators. Section 4 presents the mapping of these operators to photonic neural processors, including a C/C++ compilation workflow. Section 5 demonstrates case studies involving an MNIST classifier and EfficientNet's MBConv block. Section 6 discusses practical challenges and provides an outlook.

2 Background and Motivation

The motivation for photonic neural networks arises from the quadratic energy scaling of electronic computing with interconnect bandwidth, contrasted with the linear nature of optical operations, which are well-suited for matrix–vector multiplications (MVMs). Electronic CNN accelerators consume femtojoules to picojoules per multiply–accumulate (MAC) operation at gigahertz rates, resulting in kilowatt-scale power for large models. Photonic implementations can potentially achieve attojoule-level energy per MAC with terahertz-scale parallelism via wavelength-division multiplexing (WDM).

Early demonstrations, such as diffractive optical neural networks (DONNs) based on free-space optics, achieved high accuracy on MNIST but were limited by bulk optics. Integrated SiP architectures shifted this standard. Tait et al. [3] employed spectrum-slicing techniques to realize convolutional processing in SiP, achieving 98.6% MNIST accuracy with reduced energy consumption. For advanced architectures, primitives such as depthwise-separable convolutions have been mapped to MRRs [7], showing orders-of-magnitude efficiency gains.

A systematic framework capable of compiling CNNs directly into photonic hardware would democratize photonic ML, enabling workflows analogous to FPGA synthesis. Such a capability would target a range of applications, including edge AI, data centers, and beyond-5G communications.

3 Mathematical Representation of CNNs

At the onset of this section, it is paramount to keep cognizance that Fourier transforms and spectral filtering are physically natural operations.

CNN components are formalized as composable operators. Consider an input feature map $X \in \mathbb{R}^{H\times W\times C_{in}}$.

3.1 Convolutional Layers

A standard 2D convolution with kernel size $K_h \times K_w$, stride s, and padding p yields

$$Y_{i,j,c_o} = \sum_{m=0}^{K_h-1}\sum_{n=0}^{K_w-1}\sum_{c_i=0}^{C_{in}-1} K_{m,n,c_i,c_o}\cdot X_{s(i+m)-p,s(j+n)-p,c_i} + b_{c_o}, \tag{1}$$

where $Y \in \mathbb{R}^{H'\times W'\times C_{out}}$ is the output feature map, $K \in \mathbb{R}^{K_h\times K_w\times C_{in}\times C_{out}}$ is the kernel, $b \in \mathbb{R}^{C_{out}}$ is the bias, and H', W' are the output dimensions.

The trick here is to represent convolutions in the Fourier domain, since then convolutions become point-wise multiplication. This way we can shift convolutions into something optics is naturally good at: filtering. For circular convolution, in the Fourier domain:

$$\hat{Y}(\omega_x,\omega_y) = \hat{X}(\omega_x,\omega_y)\odot\hat{K}(\omega_x,\omega_y), \tag{2}$$

where $\hat{\cdot}$ denotes the 2D discrete Fourier transform and $\odot$ is the Hadamard (element-wise) product. The spatial output is recovered via inverse DFT: $Y = \mathcal{F}^{-1}\{\hat{Y}\}$.

For depthwise convolution:

$$Y_{i,j,c} = \sum_{m=0}^{K_h-1}\sum_{n=0}^{K_w-1} K_{m,n,c}\cdot X_{s(i+m)-p,s(j+n)-p,c}, \tag{3}$$

where the kernel is per-channel: $K \in \mathbb{R}^{K_h \times K_w \times C_{in}}$.

Pointwise convolution follows as a 1×1 convolution:

$$Y_{i,j,c_o} = \sum_{c_i=0}^{C_{in}-1} K_{0,0,c_i,c_o} \cdot X_{i,j,c_i} + b_{c_o}, \tag{4}$$

with $K \in \mathbb{R}^{1 \times 1 \times C_{in} \times C_{out}}$.

3.2 Nonlinear Activations and Pooling

Activations are applied elementwise, e.g., ReLU: $\sigma(x) = \max(0, x)$ or Swish: $\sigma(x) = x \cdot \sigma_{\text{sigmoid}}(x)$. Pooling aggregates features, e.g., max-pooling:

$$P_{i,j,c} = \max_{m,n \in [0,P-1]} X_{si+m,sj+n,c}, \tag{5}$$

or average-pooling using the mean:

$$P_{i,j,c} = \frac{1}{P^2} \sum_{m=0}^{P-1} \sum_{n=0}^{P-1} X_{si+m,sj+n,c}, \tag{6}$$

where P is the pool size.

3.3 Residual and SE Modules

Residual connections are defined as $Y = X + \mathcal{F}(X)$, where $\mathcal{F}$ denotes the residual block. For squeeze-and-excitation (SE), global average pooling yields

$$z_c = \frac{1}{HW} \sum_{i=1}^{H} \sum_{j=1}^{W} X_{i,j,c}, \tag{7}$$

followed by two fully connected layers:

$$s = \sigma\left(W_2 \delta\left(W_1 z + b_1\right) + b_2\right), \tag{8}$$

with δ as ReLU, σ as sigmoid, and rescaling

$$\tilde{X}_{i,j,c} = X_{i,j,c} \cdot s_c. \tag{9}$$

3.4 C/C++ Representation

Models are exported as structured arrays. The following pseudocode illustrates a convolutional layer:

```
struct ConvLayer {
  float kernel[K_h][K_w][C_in][C_out];
  float bias[C_out];
};

Eigen::Tensor<float, 3> ConvForward(const Eigen::Tensor<float, 3>& X, const ConvLayer& layer) {
  int H = X.dimension(0), W = X.dimension(1), Cin = X.dimension(2);
  int Cout = layer.bias.size();
  Eigen::Tensor<float, 3> Y(H, W, Cout);  // Assume stride=1, padding=0 for simplicity
  for (int i = 0; i < H; ++i) {
    for (int j = 0; j < W; ++j) {
      for (int co = 0; co < Cout; ++co) {
        float sum = layer.bias[co];
```

```
        for (int m = 0; m < K_h; ++m) {
          for (int n = 0; n < K_w; ++n) {
            for (int ci = 0; ci < Cin; ++ci) {
              sum += layer.kernel[m][n][ci][co] * X(i + m, j + n, ci);
            }
          }
        }
        Y(i, j, co) = sum;
      }
    }
  }
  return Y;
}
```

This representation enables numerical validation against frameworks such as PyTorch.

4 Photonic Neural Processors

Photonic processors exploit interference for linear operations and photodetection for nonlinearities. Key SiP components include MZIs (tunable 2×2 unitaries via phases ϕ_1, ϕ_2, with transmission matrix $\begin{pmatrix} t & r \\ r & t \end{pmatrix}$, $t = e^{i\phi_2/2}\cos(\pi/4 + \phi_1/2)$), MRRs (Lorentzian filters tuned by thermo-optics), and MMIs (passive $N \times M$ couplers via self-imaging).

4.1 Mapping CNN Operators to Photonic Primitives

CNN operators are mapped to SiP primitives such as MZIs, MRRs, and MMIs. Hybrid designs incorporate photodiodes for nonlinearities.

4.1.1 Mapping Convolutions

Fourier-domain convolution is implemented as $\mathcal{F}^{-1}[\mathcal{F}\mathbf{x} \odot \mathcal{F}\mathbf{W}]$ using star couplers for discrete Fourier transforms and phase/amplitude masks via MRR arrays. The kernel array $\mathbf{W}$ programs mask phases as $\phi_k = \arg(\mathcal{F}\mathbf{W}_k)$.

The circuit comprises an input waveguide, MMI splitter, star coupler (DFT), MRR mask array (pointwise multiply), inverse star coupler, and detector array, with a total footprint of approximately $0.5\,\mathrm{mm}^2$ per layer.

Spatial-domain convolution (e.g., 3×3 kernel) flattens patches to vectors and applies weights via GST attenuators, summed with MMI trees. The primitive consists of parallel waveguides with GST segments followed by a tree of 2×1 MMIs.

Depthwise convolution uses per-channel MRR filters, with resonances ω_c set by kernel weights. Pointwise convolution decomposes $\mathbf{W} = U\Sigma V^T$ via SVD, implementing unitaries with MZI meshes (Reck scheme) and Σ via attenuators.

The pointwise circuit includes input wavelengths (WDM), an MZI mesh of cascaded 2×2 blocks, an amplifier array for Σ, and output waveguides.

These mappings achieve low latencies ($\sim$1 ps/MAC) but incur losses (e.g., 0.5 dB/unit for MZI meshes, $1\,\mathrm{dB/Q} - \mathrm{factor}$ for MRRs). Hybrid designs use electronic readout for mitigation.

4.1.2 Pooling and Activations

Average pooling is realized via photodiode integration over bandwidth windows; max pooling uses GST-based optical switches. Activations employ hybrid square-law detection $|E|^2$ for ReLU approximation or electronic post-processing for Swish, with all-optical alternatives via Kerr nonlinearities in resonators. The primitive includes a photodiode after an MMI, followed by analog gating.

4.1.3 Residuals and SE Modules (MBConv)

Residual shortcuts use coherent addition in MMI combiners: $E_{out} = E_{main} + E_{skip}$ (phase-aligned). Squeeze-and-excitation involves global average pooling, electronic fully connected layers, and MRR channel modulators.

The circuit includes branch splitters, a main path (depthwise + projection), and an MMI adder with delay line for the skip connection. Footprints are approximately 1 mm^2 for a 32×32 MZI mesh and 0.1 mm^2/channel for MRR arrays.

4.1.4 C/C++ to Photonics Compilation Workflow

The compilation workflow proceeds as follows:

1. Export weights to structured arrays.
2. Validate inference numerically.
3. Decompose operators (e.g., kernels to Fourier coefficients or SVD angles $\phi = \cos^{-1}(U_{ij})$).
4. Simulate transfer matrices for photonic primitives.
5. Generate SiP control parameters (e.g., heater voltages for phases).
6. Deploy with hybrid electronic–photonic interface.

This ensures fidelity.

5 Case Studies

5.1 MNIST Classification with Hybrid Photonic CNN

A two-layer CNN (Conv32–ReLU–Pool–Conv64–Dense) is mapped to SiP using spectrum-slicing with MRRs for convolutional layers and photodiode pooling. Simulations achieve 98.6% accuracy at 0.1 fJ/MAC with throughput of 7.6×10^6 images/s, consistent with experimental demonstrations [3]. All-optical implementations using GST waveguides yield 91.9% accuracy at 0.53 pJ/MAC.

5.2 EfficientNet MBConv Block Mapping

The MBConv block (1×1 expansion, depthwise 3×3 convolution, Swish, squeeze–excitation, 1×1 projection, and skip) is mapped using MZI meshes for expansion/projection, MRR filters for depthwise convolution, and hybrid processing for nonlinearities and SE. Conceptual simulations indicate sub-fJ/MAC operation with picosecond latency per block.

Table 1: Photonic CNN Performance Comparison

Architecture	Mapping	Accuracy/Energy	Ref.
MNIST Hybrid CNN	MRR Spectrum-Slicing + Dense	98.6%, 0.1 fJ/MAC	[3]
All-Optical MNIST	GST Waveguides + MMI Tree	91.9%, 0.53 pJ/MAC	[4]
MBConv Conceptual	MRR + MZI Hybrid	Sub-fJ/MAC, ps latency	[7, 8]
Scaled PDONN	Partial Coherence Layers	>95% CIFAR-10	[10]

As shown in Table 1, the proposed mappings compare favorably with existing implementations.

6 Challenges and Outlook

Challenges include optical propagation losses (1 dB/cm to 3 dB/cm), mitigated by erbium-doped fiber amplifiers (adding ∼10% power overhead) or partial-coherence schemes for scalability to 100+ layers. Fabrication tolerances (<5° phase) are addressed via ML calibration. Nonlinearities remain hybrid, with fully optical Kerr-based approaches under development.

Future directions include category-theoretic formulations for operator composition, compiler frameworks for automatic embedding, and integration of partial-coherence methods for large-scale networks. The proposed framework positions SiP-based PNNs as a viable platform for EfficientNet-class models, with targeted efficiency gains of two orders of magnitude.

References

[1] A. Tsirigotis et al., "Integrated Photonic Accelerator Based on Optical Spectrum Slicing for Convolutional Neural Networks," *arXiv:2303.10357*, 2023.

[2] A. Tsirigotis et al., "Photonic Neuromorphic Accelerator for Convolutional Neural Networks," *arXiv:2405.06434*, 2024.

[3] A. Tsirigotis et al., "Photonic neuromorphic accelerator for convolutional neural networks based on an integrated reconfigurable mesh," *Commun. Eng.*, vol. 4, p. 80, 2025.

[4] M. Amiri and M. Miri, "All-optical convolutional neural network based on phase change materials in silicon photonics platform," *Sci. Rep.*, vol. 15, p. 6259, 2025.

[5] A. N. Tait et al., "Neuromorphic photonic networks using silicon photonic weight banks," *Sci. Rep.*, vol. 9, p. 9511, 2019.

[6] X. Lin et al., "All-optical machine learning using diffractive deep neural networks," *Science*, vol. 361, no. 6406, pp. 1004–1008, 2018.

[7] Y. Wei et al., "Lightweight optical neural network based on micro-ring resonator," *Opt. Express*, vol. 33, pp. 10674–10689, 2025.

[8] Z. Dong et al., "Photonic logic tensor computing beyond Tbit/s per core," *Optica*, vol. 12, pp. 1252–1260, 2025.

[9] J. Feldmann et al., "Parallel convolutional processing using an integrated silicon photonic tensor core," *Nature*, vol. 589, pp. 52–58, 2021.

[10] J. Feldmann et al., "Scaling up for end-to-end on-chip photonic neural network inference," *Light Sci. Appl.*, vol. 14, p. 328, 2025.

[11] P. Xu et al., "A Cross-Layer Optimized Silicon Photonic Neural Network Accelerator," *arXiv:2102.06960*, 2021.

[12] Y. Shen et al., "Deep learning with coherent nanophotonic circuits," *Nat. Photonics*, vol. 11, pp. 441–446, 2017.

[13] Y. Shen et al., "An integrated large-scale photonic accelerator with ultralow latency," *Nature*, vol. 640, pp. 361–367, 2025.

[14] E. Tamir et al., "Partial coherence enhances parallelized photonic computing," *Nature*, vol. 632, pp. 54–59, 2024.

[15] P. R. Prucnal et al., "A Survey on Silicon Photonics for Deep Learning," *arXiv:2101.01751*, 2021.

[16] S. Zanotto et al., "Noise-resilient and high-speed deep learning with coherent silicon photonic neural networks," *Nat. Commun.*, vol. 13, p. 5578, 2022.

[17] M. Reck et al., "Experimental realization of any discrete unitary operator," *Phys. Rev. Lett.*, vol. 73, p. 3075, 1994.

[18] M. Tan and T. LeCun, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *Proc. ICML*, 2019.